AI ENGINEER / GENAI ENGINEER

Anonymous Interview Preparation Guide

Focus: Python backend, GenAI application engineering, LLM APIs, RAG concepts, agentic workflows, AWS, data pipelines, Docker, CI/CD, observability, testing, and production troubleshooting.

Candidate profile used for preparation: 3+ years across Python backend development, AWS data engineering, APIs, cloud automation, practical LLM-enabled workflows, and a Python/FastAPI automation platform. The guide deliberately does not claim deep-learning research, production vector-database ownership, Kubernetes, LangGraph/CrewAI/AutoGen, or advanced LLM evaluation experience.

1. Target Role & Match Overview

- **Strong match:** Python, FastAPI/Flask, REST APIs, SQL, Pandas, NumPy, AWS serverless/data workflows, Docker, CI/CD, testing, monitoring, production troubleshooting.
- **Applied AI match:** LLM API integration, prompt-driven automation, structured outputs, RAG concepts, context-aware workflows, backend integration of AI capabilities.
- **Transferable strength:** building reliable data/API pipelines is directly useful for RAG ingestion, AI orchestration, evaluation pipelines, observability, retries, security, and cost controls.
- **Main gaps:** LangChain/LangGraph/LlamaIndex/CrewAI/AutoGen, vector DBs in production, reranking, formal LLM evaluation frameworks, PyTorch/deep learning, Kubernetes/vLLM/MCP, GCP/Azure AI platforms.

Interview positioning: present as an applied AI/backend engineer who can build the surrounding production system reliably, while being explicit about which GenAI components are hands-on versus conceptual/POC-level.

2. 60-Second Introduction

Template: “I am a Python engineer with 3+ years of experience across backend development, AWS data engineering, APIs, and workflow automation. In my current work I build Python and AWS pipelines using services such as Glue, Lambda, S3, Step Functions, SQS, DynamoDB, Redshift, and CloudWatch. I also have practical exposure to LLM APIs, prompt-driven automation, structured workflows, and RAG concepts. My strength is connecting AI capabilities with reliable backend systems - APIs, data preparation, monitoring, testing, and cloud deployment. I am now targeting AI Engineer roles where I can use my Python, AWS, data, and backend experience to build production-oriented GenAI applications.”

Do not say: “I am an ML researcher,” “I trained large language models,” or “I have production LangGraph/vector DB experience” unless directly verified by a project you can defend.

3. 2-Minute Introduction

“I started as a software developer building Python backend services, REST APIs, MySQL workflows, authentication, reporting, and React-integrated business applications. That gave me a strong base in API design, SQL, debugging, testing, and production support. I then moved deeper into data engineering and cloud automation, where I build AWS Glue/PySpark pipelines, Lambda and Step Functions workflows, S3/SQS processing, Oracle API integrations, monitoring, and data-quality handling.”

“Alongside that work, I have been developing practical AI automation skills: integrating LLM APIs, prompt-driven workflows, structured outputs, data preparation, and RAG concepts. I also built an independent Python/FastAPI automation platform with PostgreSQL, Redis, Docker, workers, real-time integrations, analytics, and long-running job handling. That project strengthened my system-design thinking around API orchestration, reliability, queues, monitoring, and safe automation.”

“For an AI Engineer role, I see my value in building the full application around the model: data ingestion, retrieval pipelines, API integration, guardrails, observability, testing, deployment, and failure handling. I am continuing to deepen my hands-on experience with vector databases, agent frameworks, evaluation, and model-serving patterns.”

4. Project Deep Dive - AWS Data / Automation Work

Q: Explain one production-style pipeline end to end.

Use this structure: **source -> ingestion -> transformation -> validation -> storage -> orchestration -> monitoring -> recovery**. Example: Oracle/finance source data enters through APIs or storage, Glue/PySpark performs transformations and schema mapping, Step Functions coordinates Lambda/Glue steps, S3 holds data, DynamoDB/Redshift support enrichment/storage, and CloudWatch captures logs/alerts. Explain retries, idempotency, validation, and what happens on partial failure.

Q: Why is this relevant to AI engineering?

RAG and GenAI systems need the same engineering discipline: document/data ingestion, chunking/metadata preparation, embedding jobs, index refresh, API orchestration, traceability, retries, observability, and controlled recovery. The model is only one component of the production system.

5. Project Deep Dive - Python/FastAPI Automation Platform

Q: Describe the architecture.

Use: Next.js UI -> FastAPI backend -> PostgreSQL for durable application data -> Redis/worker queue for long-running jobs -> broker/integration agent -> WebSocket/alerts for status -> Dockerized services. Emphasize API contracts, validation, queueing, state handling, monitoring, and separation of concerns.

Q: What AI-engineering lesson does this project prove?

It demonstrates orchestration of long-running automated workflows, deterministic rule execution, external integrations, async processing, operational dashboards, and safety controls. These patterns transfer directly to agent execution, tool calling, document processing, and AI job orchestration.

Q: How would you add an LLM safely?

Keep the LLM behind a service boundary, define a structured schema for outputs, validate with Pydantic, set timeouts/retries, redact sensitive inputs, add prompt/version tracking, record model metadata, and provide deterministic fallbacks when the model response is invalid or unavailable.

6. GenAI Fundamentals

Q: What is an LLM application pipeline?

A typical flow is user/input -> preprocessing/context building -> prompt or message construction -> model call -> structured validation -> tool/retrieval interaction if needed -> post-processing -> response -> logging/evaluation. Production systems also add caching, rate limits, security, observability, cost tracking, and fallbacks.

Q: Prompt engineering vs fine-tuning?

Prompt engineering changes instructions/context at inference time and is faster/cheaper to iterate. **Fine-tuning** changes model weights for a specialized behavior or domain pattern and requires curated data, evaluation, training infrastructure, and lifecycle management. Start with prompting/RAG unless evidence shows fine-tuning is needed.

Q: What are structured outputs?

The model is constrained or instructed to return a known schema such as JSON. The application validates it before use. This reduces brittle string parsing and is especially important for APIs, agents, workflow automation, and tool calls.

7. RAG - Core Concepts

Q: Explain RAG simply.

Retrieval-Augmented Generation retrieves relevant external knowledge at request time and gives that context to the LLM before generation. It improves grounding and freshness without retraining the model.

Q: Typical RAG pipeline?

Ingestion -> clean/parse -> chunk -> metadata -> embeddings -> vector index -> query embedding -> retrieve top-k -> optional filtering/reranking -> prompt assembly -> LLM answer -> citations/evaluation.

Q: What can make RAG fail?

- Poor chunking or missing metadata.
- Wrong embedding model or domain mismatch.
- Low-recall retrieval or too many irrelevant chunks.
- No reranking/filtering, weak prompts, stale index, or context-window overload.
- Evaluation only on happy-path questions instead of a representative test set.

8. RAG - Honest Gap Handling

Q: Do you have production vector-database experience?

Recommended answer: “I understand the RAG architecture and have practical experience with Python APIs, data preparation, SQL/data pipelines, and LLM integration, but I would not claim deep production ownership of a vector database yet. My transferable experience is strong on ingestion, transformation, metadata, API design, monitoring, and reliability. I am comfortable building a focused POC using a vector store and then validating retrieval quality before productionizing it.”

Q: Which vector database would you choose?

Do not pick one blindly. Explain selection criteria: scale, latency, filtering, hybrid search, managed vs self-hosted, operational burden, cost, security, cloud fit, and metadata needs. PostgreSQL with pgvector can be attractive when scale is moderate and the team already operates Postgres; specialized vector stores help when retrieval scale/features justify them.

9. Agentic AI & Tool Calling

Q: What is an AI agent?

An agent is an application loop where a model can decide actions, call tools, observe results, update state, and continue until a goal or stop condition is reached. The engineering challenge is not only planning; it is safe tool access, state, retries, timeouts, cost, observability, and deterministic boundaries.

Q: Tool calling vs normal prompting?

Normal prompting asks the model to return text/structured data. Tool calling lets the model choose a predefined function with structured arguments. The application executes the function and returns the result to the model. Tool schemas and validation are critical.

Q: How do you prevent an agent from becoming unsafe or expensive?

- Allow-list tools and validate arguments.
- Set max steps, budgets, timeouts, and rate limits.
- Require human approval for high-impact actions.
- Use idempotency keys for external side effects.
- Log every tool call and state transition; test adversarial prompts.

10. LLM Reliability, Guardrails & Evaluation

Q: What would you monitor in production?

Latency, error rate, token usage/cost, tool failures, retrieval quality, invalid structured outputs, refusal rate, fallback usage, user feedback, hallucination/grounding metrics, prompt/model version, and trace IDs across the request.

Q: How do you evaluate an LLM feature?

Create a representative labeled test set, define task-specific metrics, compare prompt/model/retrieval versions, run automated checks plus human review, and evaluate failure classes separately. For RAG, measure retrieval relevance/recall and answer faithfulness, not just answer fluency.

Q: What are guardrails?

Application controls around the model: input validation, content/security checks, schema validation, access control, tool allow-lists, sensitive-data handling, output filtering, budgets, and escalation/fallback policies.

11. Python / FastAPI Questions

Q: async vs sync in FastAPI?

Use async for I/O-bound work such as network/database calls when the libraries are async-aware. CPU-heavy work should not block the event loop; move it to workers/processes. Long-running jobs belong in a queue rather than an HTTP request lifecycle.

Q: How do you design a reliable AI API endpoint?

Pydantic request/response schemas, authentication/authorization, validation, request IDs, timeout and retry policy, structured model outputs, rate limits, safe error messages, logging/tracing, model/prompt version metadata, and fallback behavior.

Q: How would you cache LLM requests?

Only where semantics allow it. Build a stable cache key from normalized input, model/version, prompt version, retrieval context/version, and relevant parameters. Add TTL and avoid caching sensitive data without a security design.

12. SQL / Data Questions

Q: How would you deduplicate records?

Use a business key plus ordering rule, commonly ROW_NUMBER() OVER (PARTITION BY key ORDER BY updated_at DESC), then keep row_number = 1. Explain why the ordering column represents the desired canonical row.

```
SELECT * FROM ( SELECT t.*, ROW_NUMBER() OVER (PARTITION BY id ORDER BY updated_at DESC) rn FROM source t ) x WHERE rn = 1;
```

Q: How do you handle missing values?

First classify why the value is missing. Depending on semantics: reject invalid rows, keep NULL, fill with a business default, derive from other fields, or impute for analytical/ML use. Always track data-quality impact instead of blindly calling fillna().

13. AWS / Cloud Questions

Q: Lambda vs Glue?

Lambda is good for event-driven, short-duration functions and lightweight transformations. Glue is better for distributed ETL, Spark workloads, catalog integration, and larger batch processing. Use Step Functions to orchestrate multi-step workflows and CloudWatch for observability.

Q: Glue job succeeds but output is missing. What do you check?

- Input path/partition and runtime arguments.
- IAM role permissions and KMS permissions if encrypted.
- Transformation result count and filters.
- Output path, write mode, partition logic, and bookmarks.
- CloudWatch driver/executor logs and S3 object history.

Q: How do you control cloud cost?

Right-size compute, avoid unnecessary scans, partition data, use lifecycle policies, batch calls, cache where appropriate, set log retention, measure token/model cost, and choose managed services only where the operational tradeoff is justified.

14. Production Troubleshooting Scenarios

Scenario: LLM endpoint latency suddenly doubles.

Separate provider/model latency from your application. Check traces, token counts/context size, retrieval latency, rate limiting, retries, concurrent traffic, network errors, and downstream tools. Add timeout budgets and a fallback model or degraded mode where appropriate.

Scenario: RAG answers become worse after an index refresh.

Compare old/new ingestion versions, chunking, metadata, embedding model, filters, top-k, and reranker. Run a fixed retrieval evaluation set to isolate whether recall, ranking, or generation regressed. Roll back the index version if needed.

Scenario: Agent repeats the same tool call.

Inspect state and stop conditions, add step limits, detect duplicate tool-call signatures, make tool responses explicit, validate whether the model received the observation, and use idempotency for side-effecting tools.

15. System Design - Production RAG API

Prompt: Design a document-question answering system.

- **Ingestion:** upload/API -> object storage -> parsing -> chunking -> metadata -> embedding jobs -> index.
- **Serving:** API Gateway/load balancer -> FastAPI -> auth -> query processing -> retrieval -> rerank -> prompt -> LLM -> schema validation -> answer/citations.
- **State:** relational DB for users/config/audit; vector index for retrieval; cache for hot/repeat requests.
- **Async:** queue/workers for ingestion, reindexing, evaluations, and long tasks.
- **Reliability:** retries, DLQ, idempotency, timeouts, model fallback, index versioning.
- **Observability:** trace ID, prompt/model version, retrieval hits, token/cost, latency, error class, user feedback.
- **Security:** RBAC, document-level access filtering, encrypted storage, secret management, PII controls.

16. System Design - Agentic Workflow

Prompt: Design an AI agent that can read data and trigger business actions.

Separate reasoning from execution. The model receives a bounded tool catalog. Tool calls go through a policy/validation layer, then to services. Read-only tools can be automatic; high-impact write actions can require approval. Persist state and every tool call, enforce max-step/time/cost budgets, and make external writes idempotent. Use queues for long operations and retries, and add deterministic rules for compliance-critical actions.

Good phrase in interview:

“I would not let the model directly own side effects. The model proposes an action; the application validates policy, schema, permissions, and idempotency before executing it.”

17. Framework Questions - LangChain / LangGraph / LlamaIndex

Q: What if the interviewer asks which one you used?

Be precise. If your experience is limited, say: “My hands-on work has been more direct Python/API integration and workflow orchestration. I understand what these frameworks solve - prompt/tool abstractions, retrieval pipelines, and graph/state orchestration - but I would not claim deep production usage. I am comfortable learning the framework because the underlying concepts map closely to APIs, state machines, queues, and structured tool execution.”

What to learn before interview:

- One small RAG API with FastAPI + embeddings + vector store.
- One tool-calling agent with 2-3 safe tools and step limits.
- One LangGraph state graph showing retries and a human-approval node.

18. Behavioral / STAR Questions

Q: Tell me about a production issue you handled.

S: a cloud/data workflow failed or produced unexpected output. **T:** identify the root cause and restore reliable processing. **A:** check CloudWatch logs, IAM, input/output paths, payload/schema assumptions, and downstream status; reproduce safely; fix and validate data. **R:** state only verified outcome - successful recovery, reduced recurrence, or improved monitoring if true.

Q: Tell me about learning a new technology quickly.

Use a real transition such as moving from backend development into AWS data engineering. Explain how you broke the problem into a POC, learned only the required services, validated with small data, and then applied the pattern to a real workflow.

Q: What is your weakness for this role?

Use a skill gap, not a personality cliché: “My strongest hands-on experience is Python/backend/AWS and applied LLM integration. I am still building deeper production experience with vector databases, formal LLM evaluation, and agent frameworks. I address that by building focused POCs and relating them to the production engineering patterns I already use.”

19. Questions to Ask the Interviewer

- What percentage of the role is GenAI application engineering vs model/data-science work?
- Which model providers, RAG stack, vector store, and orchestration framework do you use?
- How do you evaluate retrieval quality, hallucination risk, and agent reliability before release?
- What is your deployment model: managed APIs, self-hosted inference, or both?
- How are prompts, models, embeddings, and indexes versioned and rolled back?
- What would success look like in the first 90 days for this role?

20. Final Revision Checklist

- Practice the 60-second introduction until it sounds natural, not memorized.
- Prepare one AWS pipeline story and one Python/FastAPI architecture story end to end.
- Be able to explain RAG ingestion and retrieval without framework-specific jargon.
- Know structured outputs, tool calling, guardrails, retries, timeouts, idempotency, and observability.
- Review Python async, FastAPI validation, SQL joins/window functions, Pandas handling, and AWS troubleshooting.
- Do not overclaim LangGraph, vector databases, PyTorch, Kubernetes, MCP, vLLM, GCP, or Azure AI if not hands-on.
- Before each interview, spend 60-90 minutes building/reviewing one small RAG or tool-calling POC aligned to that company JD.

Core message: “I already know how to build reliable Python APIs, cloud workflows, data pipelines, and automation. I am applying those engineering foundations to GenAI systems and can contribute strongly on the application, integration, reliability, and productionization side.”